GI 理論の数学的基盤と代替アプローチの比較検討

A Comparative Study of Mathematical Foundations and Alternative Approaches for Geometric Intelligence Theory

Research Note / Technical Memo

著者 / Author

Étale Cohomology

所属 / Affiliation

非公開 / Affiliation withheld

連絡先 / Contact

etalecohomology@proton.me

版次 / Version

Draft for open discussion

日付 / Date

2026 年 4 月 / April 2026

ライセンス / License

CC BY 4.0

概要 / Abstract

本稿は、GI (Geometric Intelligence) 理論の数学的基盤である **VAE + pullback 計量**の正当性を批判的に検証し、5 つの代替・補完的アプローチ (情報幾何学・最適輸送・拡散幾何学・Neural ODE・GPLVM) と比較する研究ノートである。対象読者は ML / データサイエンスの研究者で、微分幾何学・位相幾何学・代数幾何学を系統的に学んでいない方を想定する。必要な数学概念は自己完結的に解説する。

GI 理論の中心命題 (Proposition 2.1) が要求する 4 条件 (コンパクト性、C^k 滑らかさ、フルランク性、単射性) を整理し、数学者から提起されうる 3 つの典型的な指摘 (A: 推定計量 vs 真の計量、B: 滑らかさ / ランク、C: 位相的仮定) に対する応答テンプレートを提供する。

本稿の主要な貢献は以下である。第一に、各代替アプローチの数学的強み・弱みを技術的に比較する。第二に、**Neural ODE** が VAE + pullback 計量の 4 条件を構造的に保証する点で GI 理論の次世代基盤として最有力であることを示す。第三に、各手法を Python で実装し、外部データ非依存で再現可能な MWE (Minimum Working Example) を提供する。実装検証では、VAE ベースラインが Proposition 2.1 条件 (iii) を $\sigma_{\min} = 0.802$ で満たし、Neural ODE が訓練なしで距離相関 0.998 を達成することを示した。第四に、単一手法への依存を避け、複数手法による相互検証を行う **スタック型設計** を推奨する。

This research note critically examines the mathematical foundations of **VAE + pullback metric**, the core construction of Geometric Intelligence (GI) theory, and compares it with five alternative or complementary approaches: information geometry, optimal transport, diffusion geometry, Neural ODE, and GPLVM. The intended readers are ML / data science researchers who have not systematically studied differential, algebraic, or topological geometry; all prerequisite mathematical concepts are introduced in a self-contained manner.

We organize the four conditions required by Proposition 2.1 of GI theory (compactness, C^k smoothness, full-rank, injectivity) and provide response templates to three typical objections raised by mathematicians (A: estimated vs true metric; B: smoothness / rank; C: topological assumptions).

Main contributions: (1) a technical comparison of the mathematical strengths and weaknesses of each alternative approach; (2) the argument that **Neural ODE** is the most promising next-generation foundation for GI theory, as it structurally guarantees all four conditions of Proposition 2.1; (3) Python implementations providing reproducible MWEs (Minimum Working Examples) independent of external data, including empirical validation (the VAE baseline satisfies condition (iii) with

$\sigma_{\min} = 0.802$; Neural ODE achieves a distance correlation of 0.998 without training); (4) a recommendation for a **stacked architecture** that avoids dependence on a single method by cross-validating results across multiple approaches.

キーワード / Keywords

Riemannian manifold, pullback metric, variational autoencoder, Neural ODE, information geometry, optimal transport, diffusion maps, Gaussian process latent variable model, differential geometry, algebraic topology, Pontryagin's maximum principle, manifold learning, geometric deep learning, model-conditional inference.

GI 理論の数学的基盤と代替アプローチの比較検討

Research note / Technical memo — Draft for open discussion

ステータス: Draft | ライセンス: CC BY 4.0 | 対象: Python 3.10+

著者: Étale Cohomology 最終更新: 2026 年 4 月 対象読者: ML / データサイエンス研究者で、微分幾何学・位相幾何学・代数幾何学を系統的に学んでいない方

TL;DR

- GI (Geometric Intelligence) 理論は、社会経済データから VAE + pullback 計量でリーマン多様体を構成し、曲率・Lie 微分・測地線・最適制御を適用する枠組み。
- Proposition 2.1 の 4 条件 (コンパクト性 / C^k 滑らかさ / フルランク / 単射性) が数学的正当性の核。
- 4 条件の充足は数値検証に依存し、数学者から 3 つの典型的な指摘を受けやすい (A: 推定計量 vs 真の計量、B: 滑らかさ/ランク、C: 位相的仮定)。
- 代替 / 補完アプローチ 5 つを比較: **情報幾何・最適輸送・拡散幾何・Neural ODE・GPLVM**。
- **Neural ODE** が微分同相性を構造的に保証する点で最有力の代替基盤候補。

- 推奨は **スタック型設計**: VAE (Layer 1) + Diffusion Maps (Layer 2) + Neural ODE (Layer 3) + 形式検証 (Layer 4)。
- 全ての代替手法について Python 実装 (外部データ非依存の MWE) を併載。

---

1. GI 理論の論文概要

1.1 問題提起

GI 理論 (Étale Cohomology, 2026a, b) は、経営環境および政策環境を「データ駆動型リーマン多様体」として構成し、その上で微分幾何学の標準的な道具 (共変微分、曲率テンソル、Lie 微分、測地線、Pontryagin の最大原理) を適用することで意思決定支援インテリジェンスを生成する枠組みである。

従来の社会経済データ分析 (回帰、PCA、線形計画) はユークリッド構造を暗黙に仮定する。しかし、現実の経営・政策環境は本質的に非線形であり、同一のアクションが状態空間上の位置によって質的に異なる結果を生む。GI 理論はこの非線形性を**計量テンソルの場所依存性**として定式化する。

1.2 中心命題 (Proposition 2.1)

VAE デコーダ $f_\theta : U \subset \mathbb{R}^d \to \mathbb{R}^n$ が以下 4 条件を満たすとき、$f_\theta(U)$ は d 次元 C^k 級埋め込み部分多様体であり、pullback 計量

$$g_{ij}(z) = \sum_{a=1}^{n} \frac{\partial f_\theta^a}{\partial z^i} \frac{\partial f_\theta^a}{\partial z^j} = (J_{f_\theta}^{\top} J_{f_\theta})_{ij}$$

は U 上のリーマン計量となる。

条件	内容	実装上の確認方法
(i)	U はコンパクト	潜在空間を有界領域 (e.g. $\|z\| \le 3$) に制限
(ii)	$f_\theta \in C^k$ ($k \ge 3$)	C^∞ 活性化関数 (tanh, GELU, Softplus) を採用。ReLU 不可
(iii)	$\mathrm{rank} J_{f_\theta}(z) = d$ ($\forall z \in U$)	SVD による $\sigma_{\min}(J) > \varepsilon$ の数値検証

条件	内容	実装上の確認方法
(iv)	$f_\theta\|_U$ が単射	近似的単射性を有限サンプル上で検証

証明は Lee (2012, Theorem 4.25) の「コンパクト空間からの単射はめ込みは埋め込み」に基づく (詳細は 3 章)。

1.3 10 ステップパイプライン

1. データ収集 → 2. VAE 構成 → 3. pullback 計量 → 4. VDM による独立検証 → 5. クリストッフェル記号 → 6. 曲率テンソル / スカラー曲率 → 7. Lie 微分 / Lie 括弧 → 8. 測地線 → 9. 最適制御 (Pontryagin) → 10. 可視化・意思決定

1.4 本稿の立ち位置

本稿は GI 理論本体の拡張ではなく、その**数学的基盤の頑健性**を検証するメタ的な研究ノートである。

主要な検討事項は 2 つ:

1. VAE + pullback 計量のどこが厳密で、どこに注意が必要か
2. 同じ目的 (=「社会経済データから微分幾何学が使える多様体を構成する」) を達成する他のアプローチは何で、それぞれどのような数学的強みと弱みを持つか

---

2. 前提となる数学の自己完結的解説

この章では、GI 理論を批判的に議論するために最低限必要な数学的概念を、ML 研究者の直観語彙で説明する。証明は与えず、使用時に必要な**性質と注意事項**に焦点を当てる。形式的な取り扱いは do Carmo (1992), Lee (2012), Hatcher (2002) を参照されたい。

2.1 位相幾何学 (Topology)

2.1.1 位相空間・連続写像・同相

位相空間 $(X, \mathcal{O})$: 集合 X と開集合族 $\mathcal{O}$ の組。以下 3 公理: $\emptyset, X \in \mathcal{O}$ 、 $\mathcal{O}$ は任意合併で閉じる、有限共通部分で閉じる。

連続写像 $f : (X, \mathcal{O}_X) \to (Y, \mathcal{O}_Y)$: 開集合の逆像が開集合である写像。

同相写像 (homeomorphism): 全単射かつ f, f^{-1} がともに連続。「位相的に同じ」ことを表す同値関係を定める。

ML 直観: UMAP や t-SNE の「近傍構造の保存」は緩い意味での同相性の近似に相当する。

2.1.2 コンパクト性・ハウスドルフ性

コンパクト: 任意の開被覆が有限部分被覆を持つ。$\mathbb{R}^n$ の部分集合はコンパクト $\Leftrightarrow$ 有界閉集合 (Heine-Borel)。

ハウスドルフ: 異なる 2 点が互いに交わらない開近傍で分離できる。距離空間は常にハウスドルフ。

なぜ重要か: コンパクト空間からの連続写像は「良い性質」を持つ (例: 像もコンパクト、最大値・最小値が存在)。Lee の Theorem 4.25 の本質は「コンパクト → 閉写像 → 連続逆写像の存在」である。GI 理論が潜在空間 U のコンパクト性を要求するのはこの流れ。

2.1.3 ホモロジー・コホモロジー

特異ホモロジー $H_k(X;\mathbb{Z})$: 空間 X に含まれる「k 次元の穴」を数えるアーベル群。

- $H_0(X)$ の階数 = 連結成分数
- $H_1(X)$ の階数 = 独立なループ (1 次元の穴) の数
- $H_2(X)$ の階数 = 独立な閉曲面 (2 次元の穴) の数

コホモロジー $H^k(X;R)$: ホモロジーの双対。グローバルな不変量を局所的な情報から組み立てる際に自然に現れる。層 (sheaf) 係数のコホモロジーは、データや制約が「各点ごとに違う量 (ファイバー)」として分布する状況を扱うときに不可欠。

ML 文脈での登場例: - Topological Data Analysis (TDA): Persistent homology でデータ点集合の穴を検出 - GI ケーススタディ『偏西風の逆関数』: 飛行可能空域 B とファイバー F に対する $H^1(B,F)\neq 0$ が「通常の測地線が存在しない」ことの位相的指標

2.2 微分幾何学 (Differential Geometry)

2.2.1 可微分多様体

チャート (U,φ) : $\varphi: U \to \varphi(U) \subset \mathbb{R}^d$ は同相。**アトラス** $\{(U_\alpha,\varphi_\alpha)\}$ の遷移関数 $\varphi_\beta \circ \varphi_\alpha^{-1}$ が C^k 級のとき、C^k 級アトラス。

C^k **級可微分多様体**: 第二可算ハウスドルフ空間 + 極大 C^k アトラス。

GI 理論での $k \geq 3$ **の要求根拠**: 曲率テンソル $R^l{}_{ijk}$ は $\partial\Gamma$ を含む → Γ は ∂g を含む → 計量 g_{ij} は C^2 以上が必要 → $g_{ij} \in C^{k-1}$ なので $k \geq 3$。ReLU は C^0 で不可、C^∞ 活性化関数 (tanh, GELU, Softplus) が安全圏。

2.2.2 接空間・接束

接空間 T_pM : 点 p における滑らかな曲線の速度ベクトル全体の集合。d 次元ベクトル空間。

ベクトル場 V : 各点 p に $V_p \in T_pM$ を割り当てる滑らかな対応。**施策・介入のモデル化**として使う。

コベクトル・コ接束 T_p^*M : 接ベクトルの双対。最適制御における随伴変数 (costate) はコベクトル場。

2.2.3 リーマン計量

定義: 各点 p で正定値対称双線形形式 $g_p : T_pM \times T_pM \to \mathbb{R}$ が滑らかに変化する場。局所座標で $ds^2 = g_{ij}(z)dz^i dz^j$ 。

pullback 計量: 写像 $f: M \to N$ と N 上の計量 h に対し、$f^*h_p(u,v) := h_{f(p)}(df_p u, df_p v)$ 。ユークリッド計量の pullback は $(J_f^\top J_f)_{ij}$ となる (GI 理論の核心)。

2.2.4 接続・共変微分

曲がった空間では「異なる点の接空間が異なるベクトル空間」であるため、通常の偏微分はベクトル場の本当の変化率を与えない。**アフィン接続** ∇ が必要。

Levi-Civita 接続: リーマン多様体上で一意に存在する接続で、以下 2 条件を満たす:

- 捩れなし: $\nabla_X Y - \nabla_Y X = [X, Y]$
- 計量整合: $\nabla g = 0$

クリストッフェル記号:

$$\Gamma^k_{ij} = \frac{1}{2} g^{kl} (\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij})$$

共変微分:

$$\nabla_i V^k = \partial_i V^k + \Gamma^k_{ij} V^j$$

2.2.5 曲率テンソル

Riemann 曲率テンソル:

$$R^l{}_{ijk} = \partial_j \Gamma^l_{ik} - \partial_i \Gamma^l_{jk} + \Gamma^l_{jm} \Gamma^m_{ik} - \Gamma^l_{im} \Gamma^m_{jk}$$

リッチテンソル $\mathrm{Ric}_{ij} = R^l{}_{ilj}$ 、**スカラー曲率** $\mathrm{Scal} = g^{ij} \mathrm{Ric}_{ij}$ 。

意味の ML 直観: スカラー曲率は局所的な「発散率 / 収束率」の符号付き指標。$\mathrm{Scal} > 0$ は測地球体積が Euclid より小 (収束、安定)、$\mathrm{Scal} < 0$ は体積が大 (発散、不安定)。

2.2.6 Lie 微分と Lie 括弧

Lie 微分 $\mathcal{L}_V g$: ベクトル場 V のフローに沿った計量の変化率。

$$(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$$

$\mathcal{L}_V g = 0$ のとき V は Killing 場 (等長変換を生成)。

Lie 括弧 $[V, W]$: 2 つのベクトル場の非可換性。

$$[V, W]^k = V^i \partial_i W^k - W^i \partial_i V^k$$

GI 理論での解釈: - $\mathcal{L}_V g = 0$: 施策 V は環境の構造を保存 (状態のみ動かす) - $\mathcal{L}_V g \neq 0$: 施策が環境構造を変形 - $[V_A, V_B] \neq 0$: 施策 A, B の実行順序が結果を変える

2.2.7 測地線と最適制御

測地線方程式:

$$\ddot{\gamma}^k + \Gamma^k_{ij} \dot{\gamma}^i \dot{\gamma}^j = 0$$

Pontryagin 最大原理: 制御系 $\dot{z}^k = F^k + u^\alpha B^k_\alpha$ に対し、ハミルトニアン

$$H(z, p, u) = p_k (F^k + u^\alpha B^k_\alpha) - \ell(z, u)$$

が最適制御 u^* のもとで最大となる (一次必要条件)。随伴 p は $\dot{p}_k = -\partial H / \partial z^k$ を満たす。

2.3 代数幾何学 (Algebraic Geometry) — GI 理論に直接必要な最小限

GI 理論本体は代数幾何学の深い結果を直接使わないが、以下の概念は議論に登場する:

- **スキーム** (Hartshorne, 1977): 可換環を貼り合わせた局所環付き空間。データのカテゴリー的抽象化の極限形。
- **層 (sheaf)**: 各開集合にデータを割り当て、制限写像と整合性を持つ対応。「各地点で異なる制約 (ETOPS、保険、制裁リスト)」をモデル化するときに登場。
- **層係数コホモロジー** $H^k(X, \mathcal{F})$: 層 $\mathcal{F}$ のグローバルな情報取り出し。GI ケーススタディ 11 で $H^1(B, F)$ として使用。

本稿の 3 章以降、位相幾何と微分幾何は頻用するが、代数幾何の語彙は最小限に留める。

3. VAE + pullback 計量の正当性と 4 つの注意事項

3.1 何が数学的に正しいか

VAE デコーダ $f_\theta : Z \to X$ が滑らかなら、pullback 計量 $g_{ij} = (J^\top J)_{ij}$ は**代数的構成**として正しい。すなわち:

- 対称性: $(J^\top J)^\top = J^\top J$ ✓
- 半正定値性: $v^\top (J^\top J) v = \|Jv\|^2 \geq 0$ ✓
- J フルランクなら正定値 ✓

これに Lee (2012, Theorem 4.25) を組み合わせると、Z がコンパクトかつ f_θ が単射滑らかはめ込みならば、$f_\theta(Z)$ は埋め込み部分多様体となり、g_{ij} はリーマン計量の定義を自動的に満たす。

ここまでは **non-negotiable な数学的事実**であり、数学者の指摘は入らない。問題は「VAE が実際に Proposition 2.1 の 4 条件を満たすか」である。

3.2 注意事項 1: 滑らかさ (活性化関数の選択)

問題: ReLU は $x = 0$ で $C^0 \setminus C^1$。Pullback 計量の 2 階微分 (曲率計算に必須) が「折れ目超平面」上で未定義。

対策: 全層で C^∞ 活性化関数を使用。代表的な選択肢:

活性化関数	定義	微分	備考
tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	$1 - \tanh^2(x)$	C^∞、飽和あり
GELU	$x\Phi(x)$	解析的計算可	C^∞、SOTA で多用
Softplus	$\log(1 + e^x)$	シグモイド	C^∞、ReLU の smooth 近似
SiLU/Swish	$x\sigma(x)$	解析的計算可	C^∞

活性化関数	定義	微分	備考
ELU	x if $x \geq 0$ else $\alpha(e^x - 1)$	$\alpha = 1$ で C^1 のみ	C^∞ ではない点に注意

残余の懸念: tanh の飽和領域では $\tanh'(x) \to 0$ により有効ランクが落ちるリスク。飽和は「数学的非滑らかさ」とは別種の病理だが、条件 (iii) の数値検証に影響する。

3.3 注意事項 2: ヤコビアンのランク落ち

問題: ある z_0 で $\mathrm{rank} J_{f_\theta}(z_0) < d$ ならば、g はゼロ固有値を持ち、g^{-1} が存在せず、Γ と曲率が未定義。

対策: SVD で最小特異値 $\sigma_{\min}(J)$ を常時監視。GI ケーススタディ 11 (航空) では閾値 10^{-4} を採用、実測 $\sigma_{\min} = 0.312$ 。

数学者の突き込みどころ:

- 「全点でフルランク」をいかに離散サンプリングで保証するか (原理的には無限個の点で検証しないと厳密にはならない)
- 確率 1 で J は generic にフルランク、しかし測度ゼロの退化集合が最適化中に発現することは排除できない

実務的な落としどころ: 解析全領域で検証、閾値超過領域は “low confidence” フラグを付与 (後述の信頼度マップ)。

3.4 注意事項 3: 潜在空間の位相

問題: VAE の潜在空間は $\mathbb{R}^d$ と仮定されるが、データの真の多様体 M_0 がトーラス T^d 、球面 S^d 、双曲空間 $\mathbb{H}^d$ の場合、位相的ミスマッチ。

典型例: - 季節性を含む経済データ → S^1 因子を含む - 方位データ → S^2 ないし $SO(3)$ - 階層構造 → $\mathbb{H}^d$ (Poincaré 埋め込み; Nickel & Kiela, 2017)

対策: 用途に応じて Hyperspherical VAE (Davidson et al., 2018)、Torus VAE、Poincaré VAE (Mathieu et al., 2019) を選択。

根本的な困難: データの真の位相を事前に知らない場合、どの潜在空間を選ぶかはモデル選択の問題。Topological Data Analysis (persistent homology) による事前推定が補助となるが、有限サンプルでは完全な位相推定は困難。

3.5 注意事項 4: 推定計量と真の計量

問題: g_{ij}^{VAE} は有限データ・有限パラメータから得られる**推定値**であり、真の環境計量 g_{ij}^{true} (そもそも存在するなら) の近似に過ぎない。

数学者の指摘想定: > 「貴方が計算している曲率は、デコーダ f_θ の曲率であり、真の現象の曲率ではない」

この指摘は正しい。答えるべきは以下:

1. 社会経済現象に「真の多様体構造」が存在するかは経験的問題 (manifold hypothesis の強い版)
2. 本手法が提供するのは**データと学習されたデコーダに相対的な**幾何構造
3. したがって解析結果は「モデル条件付き推論 (model-conditional inference)」として報告すべき

緩和策: - Independent validation: VDM, Diffusion Maps など独立な手法で幾何量を推定し、一致性を確認 - Bootstrap / ensembling: ランダムシードと訓練データ再標本で推定の変動を定量化 - MC Dropout (Gal & Ghahramani, 2016): epistemic uncertainty の定量化

---

4. 代替アプローチ 5 つの技術的比較

4.1 代替 1: 情報幾何学 (Information Geometry)

4.1.1 基本構成

パラメトリック統計モデル族 $\{p(x|\theta) : \theta \in \Theta\}$ のパラメータ空間 Θ を**統計多様体**とみなす。**Fisher 情報計量**:

$$g_{ij}^{\text{Fisher}}(\theta) = \mathbb{E}_{p(x|\theta)}\left[\frac{\partial \log p}{\partial \theta^i} \cdot \frac{\partial \log p}{\partial \theta^j}\right]$$

これは Fisher 情報行列のコンポーネント。

4.1.2 数学的強み

- **自動的にリーマン計量**: p が正則 (twice differentiable in θ、identifiable) なら、g^{Fisher} は正定値対称。手動の検証が VAE より少ない。
- **統計的推論との接続**: Cramér-Rao 下界が g^{Fisher} の逆行列で表される。不偏推定量の効率が計量と直結。
- **α-接続の族**: Amari の α-接続は、$\alpha = \pm 1$ で exponential / mixture 族の双対構造を与える。Levi-Civita 接続は $\alpha = 0$ の場合。

4.1.3 VAE pullback との関係

VAE エンコーダ $q_\phi(z|x)$ の Fisher 計量 (in z space) と、デコーダ $p_\theta(x|z)$ の Fisher 計量は、理想的には互いに整合する (Chen & Murphy, 2018)。実際、pullback 計量は特定の観測モデル (例: Gaussian observation, identity covariance) の下で Fisher 計量と一致する。

つまり両者は競合ではなく、**同じ対象を異なる視点から見ている**。

4.1.4 限界

- **パラメトリックモデル族の選択が恣意的**: 社会経済データに「正しい」分布族を与えるのは非自明
- **高次元での呪い**: Fisher 行列の推定には $\mathcal{O}(d^2)$ のサンプルが必要
- **非パラメトリック拡張の難しさ**: ノンパラメトリック情報幾何は発展途上

4.2 代替 2: 最適輸送 (Optimal Transport)

4.2.1 基本構成

確率測度 μ, ν 間の Wasserstein-p 距離:

$$W_p(\mu, \nu) = \left(\inf_{\pi \in \Pi(\mu, \nu)} \int \|x - y\|^p d\pi(x, y)\right)^{1/p}$$

ここで $\Pi(\mu, \nu)$ は周辺分布が μ, ν である結合分布。

Otto calculus (Otto, 2001): 確率測度全体の空間 $\mathcal{P}_2(\mathbb{R}^n)$ は、W_2 計量の下で形式的にリーマン多様体 (無限次元)。

4.2.2 数学的強み

- **位相的前提なし**: 潜在空間の形状に関する仮定が不要
- **厳密な変分構造**: Benamou-Brenier formula により W_2^2 は運動エネルギーの最小化として再定式化可能
- **確率分布の連続変形**: 時系列の分布変化を測地線として解釈可能

4.2.3 GI 文脈での用途

- 時点間のデータ分布比較 (regime change 検出)
- 異常検知 (通常分布からの W_p 距離)
- GAN / WGAN の訓練目的関数

4.2.4 限界

- **計算コスト**: 厳密解は $\mathcal{O}(n^3 \log n)$。Sinkhorn regularization で $\mathcal{O}(n^2)$ に改善されるが依然重い
- **高次元での呪い**: サンプル複雑度が $\mathcal{O}(n^{-1/d})$ (Weed & Bach, 2019)。d 大で収束が遅い
- **微分幾何量の計算**: Wasserstein 空間上の Ricci 曲率・Lie 微分の「実用的」計算はなお研究の前線。Entropic OT の Ricci 曲率は Sturm (2006), Lott-Villani (2009) が理論、具体計算は限定的

4.3 代替 3: 拡散幾何学 (Diffusion Geometry)

4.3.1 基本構成

データ $\{x_i\}_{i=1}^N$ に対し類似度カーネル $W_{ij} = \exp(-\|x_i - x_j\|^2/\varepsilon)$ を構築。対角次数行列 $D_{ii} = \sum_j W_{ij}$ とし、マルコフ推移行列 $P = D^{-1}W$。

Diffusion Map (Coifman & Lafon, 2006): P の固有値分解 $P = \Psi \Lambda \Psi^{-1}$ の上位固有ベクトルが多様体の埋め込みを与える。

Vector Diffusion Map (Singer & Wu, 2012): 局所接空間間の最適回転 $O_{ij} \in O(d)$ でラプラシアンを拡張。接続ラプラシアンのスペクトルが multilayer 幾何情報を提供。

4.3.2 数学的強み

- **スペクトル収束定理** (Belkin & Niyogi, 2008; Singer & Wu, 2012): 仮定 A1 のもとで、$N \to \infty$, $\varepsilon \to 0$, $N\varepsilon^{d/2+1} \to \infty$ のとき、グラフラプラシアンが Laplace-Beltrami 作用素 (VDM では接続ラプラシアン) に収束。
- **ノイズ頑健性**: 拡散過程がノイズを平滑化
- **パラメトリック仮定なし**: 分布族や潜在空間の形状を仮定しない

4.3.3 GI 文脈での用途

- **独立検証**: VAE pullback 計量の spectral な独立検証 (GI 10 ステップパイプラインの Step 4)
- **intrinsic dimension 推定**: 固有値の減衰から多様体次元を推定

4.3.4 限界

- **時系列データへの自然な拡張が未成熟**: 時間発展する多様体の取り扱いは発展途上
- **大規模データのスケーラビリティ**: $N \times N$ カーネル行列と固有値分解。Nyström approximation 等は必要
- ε **の選択**: 理論的最適値 vs 実務的選択にギャップ

4.4 代替 4: Neural ODE / Normalizing Flows

4.4.1 基本構成

デコーダを常微分方程式で定義:

$$\frac{dz(t)}{dt} = f_\theta(z(t), t), \quad z(0) = z_0$$

終端値 $z(T)$ をデータ空間の点とする。訓練は随伴法:

$$\frac{da(t)}{dt} = -a(t)^\top \frac{\partial f_\theta}{\partial z}$$

4.4.2 数学的強み (ここが本命)

- **微分同相性の構造的保証**: f_θ がユニフォーム Lipschitz なら、フロー $\phi_T : z_0 \mapsto z(T)$ は C^k 微分同相。Proposition 2.1 の条件 (ii) (iii) (iv) が**自動満足**。
- **Pontryagin 随伴方程式との等価性**: Neural ODE の随伴方程式 = 最適制御の costate 方程式。訓練と制御問題が同じ数学的構造。
- **体積変化の追跡**: $\frac{d\log\det J_{\phi_t}}{dt} = \mathrm{tr}(\partial f_\theta / \partial z)$ で Jacobian 行列式が解析的に計算可能 (FFJORD; Grathwohl et al., 2019)

4.4.3 GI 文脈での位置づけ

本稿の主張: **GI 理論 Vol.3 以降の数学的基盤として Neural ODE が最も有望**。理由:

1. Proposition 2.1 の 4 条件が構造的に保証される → 数学者からの指摘が大幅減
2. 最適制御と自然に統合 (pipeline Step 9 が first-class)
3. 長期時系列シミュレーションで symplectic integrator と組み合わせ可能

4.4.4 限界

- **次元削減が直接的ではない**: Neural ODE は次元を保存。次元削減には VAE encoder との併用が必要 (Continuous Normalizing Flow)
- **計算コスト**: ODE solver のコスト。Adaptive solvers (Dormand-Prince) で緩和可能だが、VAE より重い
- **訓練の安定性**: Stiff ODE が発現すると訓練が不安定

4.5 代替 5: Gaussian Process Latent Variable Model (GPLVM)

4.5.1 基本構成

潜在空間からデータ空間への写像をガウス過程でモデル化:

$$f_d(z) \sim \mathcal{GP}(0, k(z, z')), \quad d = 1, \ldots, D$$

訓練は潜在変数 $\{z_i\}$ と GP ハイパーパラメータ θ の同時最尤推定 (Lawrence, 2005)。

4.5.2 数学的強み

- **滑らかさの構造的保証**: カーネル k が C^∞ (RBF 等) なら f は確率 1 で C^∞ (Paciorek, 2003)
- **pullback 計量の解析的計算可能性**: RBF カーネルの場合、$\mathbb{E}[g_{ij}(z)]$ が解析的に書ける (Tosi et al., 2014)
- **不確実性定量化が組み込み**: 各点の予測に事後分散が自然に付随

4.5.3 GI 文脈での用途

- **高信頼度要求領域**: 創薬、医療、金融規制対応など、モデル不確実性を明示的に扱いたい分野
- **小規模データ**: $N \lesssim 10^4$ の régime で VAE より優位な場合がある

4.5.4 限界

- **スケーラビリティ**: 厳密 GP は $\mathcal{O}(N^3)$。Sparse GP (Titsias, 2009)、SVGP (Hensman et al., 2013) で $\mathcal{O}(NM^2)$ に改善、しかし $N \gg 10^5$ ではなお重い
- **カーネル選択の依存性**: RBF 以外 (Matérn, periodic) の選択がモデル性能に影響。自動選択は難しい
- **深層 GP への拡張の複雑性**: Deep GP (Damianou & Lawrence, 2013) は表現力を高めるが、推論は変分下界の近似

---

5. Python 実装可能性の検討

本章では、4 章で議論した各アプローチを Python で実装し、その実現可能性と実装上の困難を検討する。各節のコードは外部データに依存せず、toy データで動作確認できるよう設計した。すべてのコードは CPU で動作する (GPU は必須ではない)。

5.0 共通の実験設定と環境

5.0.0 本章で参照する Python スクリプトファイル一覧

本章で解説する各 Python スクリプトは、独立したファイルとして code/ ディレクトリに配置されている。各節でファイル名を明示する。

ファイル名	対応する節	主な内容	外部依存
`code/toy_data.py`	5.0.2	toy データ生成器 (Swiss Roll, Gaussian Mixture)	numpy, torch
`code/baseline_vae.py`	5.1	VAE + pullback 計量のベースライン実装	torch
`code/info_geometry.py`	5.2	情報幾何学 (Fisher 計量の有限差分計算)	torch
`code/optimal_transport.py`	5.3	最適輸送 (Wasserstein 距離・測地線)	POT, numpy
`code/diffusion_geometry.py`	5.4	拡散幾何学 (Diffusion Map のスクラッチ実装)	numpy, scipy
`code/neural_ode.py`	5.5	Neural ODE によるフロー写像 (微分同相性の検証)	torch, torchdiffeq
`code/neural_ode_control.py`	5.5.3	Pontryagin 最大原理と Neural ODE の結合	torch, torchdiffeq
`code/gplvm.py`	5.6	Bayesian GPLVM (ガウス過程潜在変数モデル)	torch, gpytorch

すべてのスクリプトは `python <ファイル名>` で単独実行可能。`toy_data.py` は他のスクリプトから import される共通ユーティリティ。

実際の動作確認結果 (著者環境: CPU, Python 3.12, 2026 年 4 月):

- `toy_data.py` ✓ 動作確認済
- `baseline_vae.py` ✓ 動作確認済 (Proposition 2.1 条件 (iii) PASS: $\sigma_{\min} = 0.802$)
- `info_geometry.py` ✓ 動作確認済 (Fisher 計量の正定値性確認)
- `diffusion_geometry.py` ✓ 動作確認済 (Swiss Roll で固有値スペクトルが減衰)
- `neural_ode.py` ✓ 動作確認済 (**訓練なしで距離相関 0.998**, $\det(J) > 0$ 全点)
- `optimal_transport.py` / `neural_ode_control.py` / `gplvm.py` — 構文検証済 (外部依存パッケージのインストール後に実行可)

5.0.1 依存パッケージ

```
# 基盤
pip install torch==2.2.0 numpy==1.26.4 scipy==1.12.0 scikit-learn==1.4.0
```

```
# 代替アプローチ向け
pip install torchdiffeq==0.2.3        # Neural ODE
pip install POT==0.9.1                # Optimal Transport
pip install pydiffmap==0.2.0.1        # Diffusion Maps
pip install gpytorch==1.11            # GP / GPLVM
pip install geomstats==2.7.0          # Information Geometry (一部)

# 可視化・補助
pip install matplotlib==3.8.2
```

Python 3.10 以上を前提とする。バージョンは 2026 年 4 月時点での安定版を例示。

5.0.2 共通の toy データ生成器

以下のコードを toy_data.py として保存し、各節で再利用する。

ファイル: code/toy_data.py

```python
# toy_data.py
import numpy as np
import torch

def make_swiss_roll(n_samples=1000, noise=0.05, seed=0):
    """
    3D Swiss Roll: 2D 多様体が 3D に埋め込まれた古典的 toy データ。
    GI 理論の検証として最適: 真の内在次元 = 2、曲率は場所依存。

    Returns:
        X: (n_samples, 3) データ点
        t: (n_samples,) 真の内在パラメータ (後で色付けに使う)
    """
    rng = np.random.default_rng(seed)
    t = 1.5 * np.pi * (1 + 2 * rng.random(n_samples))
    h = 21 * rng.random(n_samples)
    x = t * np.cos(t)
    y = h
    z = t * np.sin(t)
    X = np.stack([x, y, z], axis=1)
    X += noise * rng.standard_normal(X.shape)
    X = X.astype(np.float32)
    return X, t.astype(np.float32)


def make_gaussian_mixture(n_samples=1000, n_components=3, dim=5, seed=0):
    """
```

```
    高次元ガウス混合: 連結性が明確でない設定。
    Regime change 検出やクラスタ分離の検証用。
    """
    rng = np.random.default_rng(seed)
    means = rng.standard_normal((n_components, dim)) * 3
    covs = [np.eye(dim) * 0.5 for _ in range(n_components)]
    labels = rng.integers(0, n_components, n_samples)
    X = np.zeros((n_samples, dim))
    for i, label in enumerate(labels):
        X[i] = rng.multivariate_normal(means[label], covs[label])
    return X.astype(np.float32), labels


def to_torch(X, device="cpu", dtype=torch.float32):
    return torch.tensor(X, dtype=dtype, device=device)


if __name__ == "__main__":
    X, t = make_swiss_roll(n_samples=500)
    print(f"Swiss Roll: X.shape = {X.shape}, X.dtype = {X.dtype}")
    print(f"Range per dim: {X.min(0)} to {X.max(0)}")
```

5.1 ベースライン: VAE + pullback 計量

GI 理論の標準実装。以降の比較基準となる。

ファイル: code/baseline_vae.py

```
# baseline_vae.py
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.autograd.functional import jacobian

class SmoothVAE(nn.Module):
    """
    Proposition 2.1 の条件 (ii) を保証するため tanh を採用。
    潜在次元 d と観測次元 n を指定。
    """
    def __init__(self, input_dim: int, latent_dim: int, hidden: int = 6
4):
        super().__init__()
        self.latent_dim = latent_dim
        self.enc = nn.Sequential(
            nn.Linear(input_dim, hidden), nn.Tanh(),
            nn.Linear(hidden, hidden),    nn.Tanh(),
        )
```

```
        self.fc_mu     = nn.Linear(hidden, latent_dim)
        self.fc_logvar = nn.Linear(hidden, latent_dim)
        self.dec = nn.Sequential(
            nn.Linear(latent_dim, hidden), nn.Tanh(),
            nn.Linear(hidden, hidden),     nn.Tanh(),
            nn.Linear(hidden, input_dim),
        )

    def encode(self, x):
        h = self.enc(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def decode(self, z):
        return self.dec(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        std = torch.exp(0.5 * logvar)
        z = mu + std * torch.randn_like(std)
        return self.decode(z), mu, logvar


def vae_elbo(x_recon, x, mu, logvar, beta=1.0):
    recon = F.mse_loss(x_recon, x, reduction="sum")
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon + beta * kl


def train_vae(model, X, epochs=500, lr=1e-3, device="cpu"):
    model.to(device).train()
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    X = X.to(device)
    for ep in range(epochs):
        opt.zero_grad()
        x_recon, mu, logvar = model(X)
        loss = vae_elbo(x_recon, X, mu, logvar)
        loss.backward()
        opt.step()
        if (ep + 1) % 100 == 0:
            print(f"[VAE] epoch {ep+1:4d} | loss = {loss.item():.2f}")
    return model


def pullback_metric(decoder, z):
    """
    g_{ij}(z) = J^T J at a single point z (shape: (d,)).
```

```
    Returns (d, d) torch.Tensor.
    """
    z = z.detach().requires_grad_(True)
    J = jacobian(decoder, z, create_graph=False)   # (input_dim, d)
    return J.T @ J


def check_proposition_2_1(decoder, Z_samples, sigma_threshold=1e-4):
    """
    Proposition 2.1 の条件 (iii) の数値検証。
    Z_samples: (N, d) 潜在空間サンプル点。
    """
    sigmas_min = []
    for z in Z_samples:
        J = jacobian(decoder, z, create_graph=False)
        S = torch.linalg.svdvals(J)
        sigmas_min.append(S.min().item())
    sigmas_min = torch.tensor(sigmas_min)
    passed = (sigmas_min > sigma_threshold).all().item()
    return {
        "passed": bool(passed),
        "sigma_min_global": sigmas_min.min().item(),
        "sigma_min_mean": sigmas_min.mean().item(),
        "n_violations": int((sigmas_min <= sigma_threshold).sum().item()),
    }


if __name__ == "__main__":
    from toy_data import make_swiss_roll, to_torch
    X_np, _ = make_swiss_roll(n_samples=1000, seed=0)
    X = to_torch(X_np)
    vae = SmoothVAE(input_dim=3, latent_dim=2, hidden=64)
    train_vae(vae, X, epochs=500)

    # ランク検証
    with torch.no_grad():
        mu, _ = vae.encode(X[:100])
    Z = mu.detach()
    report = check_proposition_2_1(vae.decode, Z)
    print("Proposition 2.1 (iii):", report)

    # 1 点での pullback 計量
    z0 = Z[0]
    g = pullback_metric(vae.decode, z0)
    print("g at z_0:\n", g)
    print("eigenvalues:", torch.linalg.eigvalsh(g))
```

動作確認の観点: - `passed: True` かどうか (条件 iii が数値的に満たされるか) - g の固有値がすべて正かつ桁が揃っているか (条件数 $\kappa(g)$ が小さいか) - toy Swiss Roll (d=2, n=3) という素直なケースで成立することを確認

実務的な落とし穴: - `jacobian` はバッチ化されていない。大量の z で呼ぶと遅い。`torch.func.vmap(torch.func.jacrev(decoder))` での並列化を推奨 - ReLU に差し替えて動かすと条件 (iii) が violation しうる (折れ目近傍で特異値が落ちる)

5.2 情報幾何学の実装

5.2.1 実装戦略

Fisher 情報計量の計算には「統計モデル族の明示」が必要。ここでは最も扱いやすい**多変量ガウシアン族**を例に取る:

$$p(x|\theta) = \mathcal{N}(x; \mu(\theta), \Sigma(\theta))$$

多変量ガウシアンの Fisher 情報行列は解析的に既知:

$$g_{ij}^{\text{Fisher}} = \frac{\partial \mu}{\partial \theta^i}^{\top} \Sigma^{-1} \frac{\partial \mu}{\partial \theta^j} + \frac{1}{2} \mathrm{tr}\left(\Sigma^{-1} \frac{\partial \Sigma}{\partial \theta^i} \Sigma^{-1} \frac{\partial \Sigma}{\partial \theta^j} \right)$$

5.2.2 モジュール: geomstats

ファイル: `code/info_geometry.py`

```python
# info_geometry.py
import numpy as np
import torch

def fisher_metric_mvn(mean_fn, cov_fn, theta, eps=1e-4):
    """
    多変量ガウシアン族 N(mu(theta), Sigma(theta)) の Fisher 計量を有限差分で。

    Args:
        mean_fn:  theta -> mu (torch.Tensor, shape=(D,))
        cov_fn:   theta -> Sigma (torch.Tensor, shape=(D, D))
        theta:    (d,) torch.Tensor

    Returns:
        g: (d, d) Fisher 計量
    """
    d = theta.shape[0]
    theta = theta.detach().clone().requires_grad_(True)
```

```
    mu = mean_fn(theta)
    Sigma = cov_fn(theta)
    Sigma_inv = torch.linalg.inv(Sigma)

    # d mu / d theta^i
    dmu = torch.stack([
        torch.autograd.grad(mu.sum(), theta, create_graph=True, retain_gr
aph=True)[0]
        if False else _finite_diff_vec(mean_fn, theta, i, eps)
        for i in range(d)
    ])  # (d, D)

    # d Sigma / d theta^i
    dSigma = torch.stack([
        _finite_diff_mat(cov_fn, theta, i, eps)
        for i in range(d)
    ])  # (d, D, D)

    g = torch.zeros(d, d)
    for i in range(d):
        for j in range(d):
            # mean term
            term_mean = dmu[i] @ Sigma_inv @ dmu[j]
            # covariance term
            M = Sigma_inv @ dSigma[i] @ Sigma_inv @ dSigma[j]
            term_cov = 0.5 * torch.trace(M)
            g[i, j] = term_mean + term_cov
    return g


def _finite_diff_vec(fn, theta, i, eps):
    t_plus = theta.clone(); t_plus[i] += eps
    t_minus = theta.clone(); t_minus[i] -= eps
    return (fn(t_plus) - fn(t_minus)) / (2 * eps)


def _finite_diff_mat(fn, theta, i, eps):
    return _finite_diff_vec(fn, theta, i, eps)


if __name__ == "__main__":
    # toy: 2D ガウシアン族、theta = (mu_1, mu_2)
    def mean_fn(theta):
        return theta  # mu = theta
    def cov_fn(theta):
        # 共分散が位置依存: Sigma = I + 0.1 * theta[0]^2 * I
```

```
        return (1.0 + 0.1 * theta[0]**2) * torch.eye(2)

    theta = torch.tensor([1.0, -0.5])
    g = fisher_metric_mvn(mean_fn, cov_fn, theta)
    print("Fisher metric at theta = (1, -0.5):\n", g)
    print("eigenvalues:", torch.linalg.eigvalsh(g))
```

5.2.3 実装可能性の所見

- ◯ **解析的モデルに対しては実装容易**: ガウシアン、指数分布族なら Fisher 情報行列は closed form または半 closed form
- △ **データ駆動では困難**: 真の $p(x)$ を知らない状況で「統計モデル族」をどう選ぶかが恣意的
- ◯ **ライブラリ**: `geomstats` が Fisher 計量・natural gradient を提供。ただし製品品質はまだ発展途上

結論: パラメトリック生成モデル (Flow, GAN, VAE) の**潜在空間**上では Fisher 計量を計算可能。純粋なデータ駆動の非パラメトリック設定では実装困難。

5.3 最適輸送の実装

5.3.1 実装戦略

POT (Python Optimal Transport) を使う。W_2 距離の計算と Sinkhorn regularization の両方をサポート。

5.3.2 基本例

ファイル: `code/optimal_transport.py`

```
# optimal_transport.py
import numpy as np
import ot

def w2_distance(X, Y):
    """
    経験分布間の Wasserstein-2 距離 (厳密).
    X, Y: (n, d), (m, d) numpy arrays.
    """
    n, m = len(X), len(Y)
    a = np.ones(n) / n
    b = np.ones(m) / m
    M = ot.dist(X, Y, metric="sqeuclidean")  # (n, m)
    W2_sq = ot.emd2(a, b, M)
    return float(np.sqrt(W2_sq))
```

```python
def sinkhorn_w2(X, Y, reg=0.1):
    """
    エントロピー正則化付き Wasserstein-2. 大規模データ向け高速化.
    """
    n, m = len(X), len(Y)
    a = np.ones(n) / n
    b = np.ones(m) / m
    M = ot.dist(X, Y, metric="sqeuclidean")
    return float(np.sqrt(ot.sinkhorn2(a, b, M, reg=reg)))


def wasserstein_geodesic(X, Y, t_values):
    """
    McCann interpolation: μ_t = ((1-t) T_id + t T_*) # μ
    離散分布での 2-OT 最適カップリング T を使って、時点 t での中間分布を返す.
    """
    a = np.ones(len(X)) / len(X)
    b = np.ones(len(Y)) / len(Y)
    M = ot.dist(X, Y, metric="sqeuclidean")
    G = ot.emd(a, b, M)   # (n, m) 最適輸送計画
    # 各 x_i の輸送先を重み付き平均で
    # (離散近似: barycentric projection)
    T_X = (G @ Y) / a[:, None]   # (n, d)
    interpolations = []
    for t in t_values:
        mu_t = (1 - t) * X + t * T_X
        interpolations.append(mu_t)
    return interpolations


if __name__ == "__main__":
    np.random.seed(0)
    X = np.random.randn(200, 3)
    Y = np.random.randn(200, 3) + np.array([2.0, 0.0, 0.0])
    print(f"W_2(X, Y) exact   = {w2_distance(X, Y):.4f}")
    print(f"W_2(X, Y) Sinkhorn = {sinkhorn_w2(X, Y, reg=0.1):.4f}")

    t_vals = [0.0, 0.25, 0.5, 0.75, 1.0]
    geo = wasserstein_geodesic(X, Y, t_vals)
    print("Geodesic samples:", [g.mean(0) for g in geo])
```

5.3.3 Lie 微分・スカラー曲率の計算可能性

Wasserstein 空間上の **Ricci 曲率 ≥ K** のような lower bound は Sturm-Lott-Villani 理論で定義されているが、「各点で Ricci テンソルを数値的に計算する」直接的な手続きは確立されていない。

実務的な代替: - Entropic OT debiasing により局所曲率を近似 (Feydy et al., 2019) - Barycentric embedding で euclidean 空間に埋め込み、そこで古典的に微分幾何を

5.3.4 所見

- ◎ **二時点分布比較**: 時系列のレジーム変化検出に理想的
- ◯ **計算基盤**: POT は成熟。Sinkhorn で $\mathcal{O}(n^2)$ までスケール
- △ **微分幾何量**: Lie 微分・Ricci テンソルの直接計算は研究前線。GI 理論の core 計算には不向き
- ◎ **補助ツール**: GI pipeline の時系列比較コンポーネントとして強力

結論: GI 理論の「代替」ではなく「補完」として最適。時点間の regime detection、異常検知に適用すべき。

5.4 拡散幾何学の実装

5.4.1 実装戦略

`pydiffmap` は Coifman-Lafon の diffusion map を提供。ここでは基本的な使い方と、GI 理論の pullback 計量の独立検証としての用法を示す。

5.4.2 実装

ファイル: `code/diffusion_geometry.py`

```
# diffusion_geometry.py
import numpy as np
from scipy.spatial.distance import cdist
from scipy.sparse.linalg import eigsh

def diffusion_map_scratch(X, n_components=2, epsilon=None, alpha=1.0):
    """
    Diffusion map をゼロから実装。alpha=1 で Laplace-Beltrami に収束 (Coifm
an-Lafon).

    Args:
        X: (N, D)
        n_components: 埋め込み次元
        epsilon: カーネル幅 (None なら median heuristic)
```

```
        alpha: 正規化指数 ∈ [0, 1]

    Returns:
        embedding: (N, n_components)
        eigenvalues: (n_components + 1,)
    """
    N = len(X)
    D2 = cdist(X, X, metric="sqeuclidean")
    if epsilon is None:
        epsilon = np.median(D2[D2 > 0])
    K = np.exp(-D2 / epsilon)

    # alpha-normalization
    q = K.sum(axis=1)
    K_alpha = K / np.outer(q**alpha, q**alpha)

    # Markov normalization
    d = K_alpha.sum(axis=1)
    P = K_alpha / d[:, None]

    # 対称化版 (数値安定): A = D^{1/2} P D^{-1/2}
    # A の固有値 = P の固有値、対称なので eigsh 使用可
    A = (d[:, None] ** 0.5) * P / (d[None, :] ** 0.5)
    A = 0.5 * (A + A.T)  # 対称化 (数値誤差吸収)

    # 上位 k+1 固有対 (最大固有値 1 を除く)
    eigvals, eigvecs = eigsh(A, k=n_components + 1, which="LM")
    idx = np.argsort(-eigvals)
    eigvals = eigvals[idx]
    eigvecs = eigvecs[:, idx]

    # 対応する P の右固有ベクトルへ変換
    psi = eigvecs / (d[:, None] ** 0.5)

    # 最初の固有ベクトル (trivial, 定数) を除外
    embedding = psi[:, 1:n_components + 1] * eigvals[1:n_components + 1]
    return embedding, eigvals


def compare_pullback_vs_diffusion(X, vae, n_components=2):
    """
    VAE pullback 距離と diffusion 距離の相関を測定。
    高い相関 → 二手法が一致した幾何を抽出。
    """
    import torch
```

```python
    # VAE encoder での埋め込み
    with torch.no_grad():
        mu, _ = vae.encode(torch.tensor(X, dtype=torch.float32))
    Z_vae = mu.numpy()

    # Diffusion map での埋め込み
    Z_diff, _ = diffusion_map_scratch(X, n_components=n_components)

    # ペア距離を比較 (subsample for tractability)
    idx = np.random.choice(len(X), 200, replace=False)
    D_vae  = cdist(Z_vae[idx],  Z_vae[idx])
    D_diff = cdist(Z_diff[idx], Z_diff[idx])
    corr = np.corrcoef(D_vae.ravel(), D_diff.ravel())[0, 1]
    return corr


if __name__ == "__main__":
    from toy_data import make_swiss_roll
    X, t = make_swiss_roll(n_samples=800, noise=0.05)
    emb, eigvals = diffusion_map_scratch(X, n_components=2)
    print("Diffusion map embedding shape:", emb.shape)
    print("Top 5 eigenvalues:", eigvals[:5])
    # emb[:, 0] vs true t で単調性を確認 (Swiss Roll の場合)
    from scipy.stats import spearmanr
    rho, _ = spearmanr(emb[:, 0], t)
    print(f"Spearman rank correlation with true t: {rho:.3f}")
```

5.4.3 所見

- ◎ **VDM 独立検証**: GI 10 ステップパイプラインの Step 4 として適切
- ○ **次元推定**: 固有値の gap spectrum で intrinsic dimension を推定
- △ **時系列データ**: 時間発展する多様体には dynamic diffusion map が必要、成熟度に欠ける
- △ **スケーラビリティ**: $N > 10^4$ で $N \times N$ カーネル行列が重い。Nyström 近似が必要

結論: GI pipeline の**独立検証コンポーネント**として採用すべき。完全な代替にはならない (Lie 微分等の計算が困難)。

5.5 Neural ODE の実装 (本命候補)

5.5.1 実装戦略

torchdiffeq を使う。VAE デコーダを Neural ODE フローで置き換え、微分同相性を構造的に保証する。

5.5.2 実装

ファイル: code/neural_ode.py

```
# neural_ode.py
import torch
import torch.nn as nn
from torchdiffeq import odeint_adjoint as odeint

class ODEFunc(nn.Module):
    """ODE の右辺 f_theta(z, t). 全層 tanh で C^infty 級."""
    def __init__(self, dim: int, hidden: int = 64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(dim + 1, hidden), nn.Tanh(),
            nn.Linear(hidden, hidden),   nn.Tanh(),
            nn.Linear(hidden, dim),
        )

    def forward(self, t, z):
        # t を一緒に入力することで time-dependent dynamics を許容
        tz = torch.cat([z, t.expand(z.shape[0], 1)], dim=-1)
        return self.net(tz)


class NeuralODEFlow(nn.Module):
    """
    Neural ODE によるフロー写像。
    z_0 -> z_T は C^infty 微分同相 (f_theta が Lipschitz なら).
    """
    def __init__(self, dim: int, hidden: int = 64, t_end: float = 1.0):
        super().__init__()
        self.ode_func = ODEFunc(dim, hidden)
        self.t = torch.tensor([0.0, t_end])

    def forward(self, z0):
        # z0: (B, dim)
        traj = odeint(self.ode_func, z0, self.t, method="dopri5", rtol=1e
-5, atol=1e-5)
        return traj[-1]  # z(T)
```

```python
class CNFDecoder(nn.Module):
    """
    Continuous Normalizing Flow 風デコーダ。
    潜在空間 Z と観測空間 X の次元が同じ場合に Neural ODE を直接使える。
    異次元の場合は後述の "augmented" 構成。
    """
    def __init__(self, dim: int, hidden: int = 64):
        super().__init__()
        self.flow = NeuralODEFlow(dim, hidden)

    def forward(self, z):
        return self.flow(z)


def verify_diffeomorphism(decoder: CNFDecoder, n_points: int = 100, d: in
t = 2):
    """
    Neural ODE フローが微分同相であることを数値検証。
    - 滑らかさ: loss.backward() で 2 階微分が計算できる
    - 単射性: 異なる z が異なる出力
    - フルランク: ヤコビ行列のフルランク
    """
    z = torch.randn(n_points, d, requires_grad=True)
    x = decoder(z)
    # 出力の distinctness
    dists_z = torch.cdist(z, z)
    dists_x = torch.cdist(x, x)
    # 同一 z を排除して相関
    mask = ~torch.eye(n_points, dtype=torch.bool)
    corr = torch.corrcoef(torch.stack([dists_z[mask], dists_x[mask]]))[0,
 1]

    # フルランクの検証: 複数点で Jacobian SVD
    from torch.autograd.functional import jacobian
    sigmas = []
    for i in range(min(10, n_points)):
        J = jacobian(lambda z_: decoder(z_.unsqueeze(0)).squeeze(0), z
[i])
        S = torch.linalg.svdvals(J)
        sigmas.append(S.min().item())

    return {
        "distance_correlation": float(corr.item()),
```

```
        "sigma_min_mean": float(sum(sigmas) / len(sigmas)),
        "sigma_min_min":  float(min(sigmas)),
    }


if __name__ == "__main__":
    torch.manual_seed(0)
    decoder = CNFDecoder(dim=2, hidden=64)
    # 訓練なしでも微分同相性は構造的に保証される
    report = verify_diffeomorphism(decoder, n_points=50, d=2)
    print("Diffeomorphism verification:", report)

    # 明示的にヤコビ行列式の符号が保存されるか確認
    from torch.autograd.functional import jacobian
    z = torch.randn(5, 2)
    for i, z_i in enumerate(z):
        J = jacobian(lambda z_: decoder(z_.unsqueeze(0)).squeeze(0), z_i)
        det = torch.linalg.det(J).item()
        print(f"  z_{i}: det(J) = {det:.4f} (same sign ⇒ orientation-pre
serving)")
```

5.5.3 Pontryagin 最大原理との結合

ファイル: code/neural_ode_control.py

```
# neural_ode_control.py
import torch
import torch.nn as nn
from torchdiffeq import odeint_adjoint as odeint

class ControlledSystem(nn.Module):
    """
    dz/dt = F(z) + u(t) * B(z) の形の affine 制御系.
    Pontryagin 最大原理の下で最適 u*(t) を求める.
    """
    def __init__(self, F_net, B_net, u_net):
        super().__init__()
        self.F = F_net    # drift
        self.B = B_net    # control matrix
        self.u = u_net    # control policy u(t, z)

    def forward(self, t, z):
        F_val = self.F(z)
        B_val = self.B(z)
        u_val = self.u(torch.cat([z, t.expand(z.shape[0], 1)], dim=-1))
        # u_val: (B, m), B_val: (B, d, m) -> drift contribution (B, d)
        return F_val + torch.einsum("bdm,bm->bd", B_val, u_val)
```

```
def pontryagin_adjoint_loss(system, z0, z_target, cost_fn, t_span=(0.0, 1.
0)):
    """
    シンプルな trajectory optimization:
    end-point cost + running cost を最小化する u を学習.
    訓練で自動的に随伴変数 (costate) が計算される。
    """
    t = torch.tensor(list(t_span))
    traj = odeint(system, z0, t, method="dopri5")
    z_final = traj[-1]
    # End-point cost
    J_terminal = ((z_final - z_target) ** 2).sum()
    # Running cost (quadratic in control: 積分近似)
    J_running = 0.0
    n_eval = 10
    for k in range(n_eval):
        tk = torch.tensor(t_span[0] + k * (t_span[1] - t_span[0]) / n_eva
l)
        zk = odeint(system, z0, torch.tensor([t_span[0], tk.item()]))[-1]
        uk = system.u(torch.cat([zk, tk.expand(zk.shape[0], 1)], dim=-1))
        J_running = J_running + 0.01 * (uk ** 2).sum() / n_eval
    return J_terminal + J_running
```

5.5.4 所見

- ◎ **微分同相性の構造保証**: Proposition 2.1 の条件 (ii) (iii) (iv) が自動満足。**数学者の指摘を最も強く防げる**
- ◎ **Pontryagin との統合**: 訓練フレームワークそのものが最適制御と同一
- ◯ **実装成熟度**: `torchdiffeq` は広く使われ、adjoint 法がデフォルト
- △ **計算コスト**: ODE solver が遅い。`method="rk4"` で固定ステップ化すれば高速化可能
- △ **次元変化**: 次元削減には encoder を別途用意する必要がある (Latent ODE; Rubanova et al., 2019)

結論: GI 理論 Vol.3 の数学的基盤候補として最有力。VAE を Continuous Normalizing Flow で置き換えることで、Proposition 2.1 の 4 条件が構造的に保証される。

5.6 GPLVM の実装

5.6.1 実装戦略

`gpytorch` の `BayesianGPLVM` を使う。

5.6.2 実装

ファイル: code/gplvm.py

```
# gplvm.py
import torch
import numpy as np
import gpytorch
from gpytorch.models.gplvm.bayesian_gplvm import BayesianGPLVM
from gpytorch.models.gplvm.latent_variable import VariationalLatentVariab
le
from gpytorch.means import ZeroMean
from gpytorch.kernels import RBFKernel, ScaleKernel
from gpytorch.variational import VariationalStrategy, CholeskyVariational
Distribution
from gpytorch.priors import NormalPrior


class MyGPLVM(BayesianGPLVM):
    def __init__(self, n, data_dim, latent_dim, n_inducing=25):
        batch_shape = torch.Size([data_dim])
        inducing_inputs = torch.randn(data_dim, n_inducing, latent_dim)
        q_f = CholeskyVariationalDistribution(n_inducing, batch_shape=bat
ch_shape)
        q_u = VariationalStrategy(self, inducing_inputs, q_f, learn_induc
ing_locations=True)
        X_init = torch.nn.Parameter(torch.randn(n, latent_dim))
        prior_x = NormalPrior(torch.zeros(n, latent_dim), torch.ones(n, l
atent_dim))
        X = VariationalLatentVariable(n, data_dim, latent_dim, X_init, pr
ior_x)
        super().__init__(X, q_u)
        self.mean_module = ZeroMean(batch_shape=batch_shape)
        self.covar_module = ScaleKernel(RBFKernel(batch_shape=batch_shape,
 ard_num_dims=latent_dim),
                                        batch_shape=batch_shape)

    def forward(self, X):
        mean_x = self.mean_module(X)
        covar_x = self.covar_module(X)
        return gpytorch.distributions.MultivariateNormal(mean_x, covar_x)


def train_gplvm(Y, latent_dim=2, n_iter=500, lr=0.01):
    Y = torch.tensor(Y, dtype=torch.float32)
    n, data_dim = Y.shape
    model = MyGPLVM(n, data_dim, latent_dim)
```

```
    likelihood = gpytorch.likelihoods.GaussianLikelihood(batch_shape=torc
h.Size([data_dim]))
    mll = gpytorch.mlls.VariationalELBO(likelihood, model, num_data=n)
    opt = torch.optim.Adam([
        {"params": model.parameters()},
        {"params": likelihood.parameters()},
    ], lr=lr)
    model.train(); likelihood.train()
    for it in range(n_iter):
        opt.zero_grad()
        sample = model.sample_latent_variable()
        output = model(sample)
        loss = -mll(output, Y.T).sum()
        loss.backward()
        opt.step()
        if (it + 1) % 100 == 0:
            print(f"[GPLVM] iter {it+1:4d} | loss = {loss.item():.2f}")
    return model, likelihood


def analytic_pullback_rbf(model, Z):
    """
    RBF カーネルの場合、pullback 計量の期待値は解析的 (Tosi et al. 2014).
    E[g_ij(z)] = sum_d sigma_f^2 * (1/ell_i ell_j) * E[f_d(z) gradient te
rms]
    ここでは簡略化: 事後分散を使って数値的に近似.
    """
    # 実装簡略: deterministic posterior mean のヤコビアンを使う
    from torch.autograd.functional import jacobian
    def mean_fn(z):
        Z_batch = z.unsqueeze(0)  # (1, latent_dim)
        model.eval()
        with torch.no_grad(): pass
        # Posterior mean at Z_batch
        # Note: gpytorch の BayesianGPLVM はこの用途には追加実装が必要
        raise NotImplementedError("gpytorch BayesianGPLVM の posterior me
an API 要整備")

if __name__ == "__main__":
    from toy_data import make_swiss_roll
    X, _ = make_swiss_roll(n_samples=200)
    model, lik = train_gplvm(X, latent_dim=2, n_iter=300)
    print("Trained GPLVM. Latent locations shape:",
          model.X.q_mu.shape if hasattr(model.X, "q_mu") else "n/a")
```

5.6.3 所見

- ◎ **不確実性定量化**: 各点の予測分散が自然に得られる
- ◎ **滑らかさの保証**: RBF カーネルなら確率 1 で C^∞
- △ **スケーラビリティ**: $N > 10^4$ では sparse approximation 必須
- △ **API 成熟度**: `gpytorch` の `BayesianGPLVM` は研究用途寄り。pullback 計量の直接 API は未整備
- △ **深層 GP**: 表現力を高めるなら Deep GP が必要だが、実装複雑度が跳ね上がる

結論: 高信頼度要求 / 小規模データ用途での有力候補。GI 理論の**特定のユースケース (医療・創薬・規制対応)** で採用候補。

5.7 実装比較の横断表

手法	実装成熟度	計算コスト	曲率計算	Lie 微分計算	最適制御統合	数学的保証
VAE + pullback	◎ (PyTorch)	中	○ (autograd)	○ (autograd)	△ (別途必要)	△ (4 条件要検証)
情報幾何学 (パラメトリック)	○ (geomstats)	低〜中	○ (解析的 if 簡単)	○	△	◎
最適輸送	◎ (POT)	中〜高	△ (研究前線)	△	△	◎
拡散幾何	◎ (pydiffmap)	中 (N^2)	△ (スペクトル経由)	△	△	○
Neural ODE	◎ (torchdiffeq)	中〜高	○ (autograd)	○	◎ (随伴 = costate)	◎ (微分同相)
GPLVM	○ (gpytorch)	高 (N^3)	○ (Tosi 2014)	○	△	◎ (RBF ならば)

凡例: ◎ 強く推奨 / ○ 実用可 / △ 要注意 or 発展途上

5.8 推奨アーキテクチャ: スタック型アプローチ

本章の所見を統合して、以下のスタック型 GI 実装を推奨する:

```
┌──────────────────────────────────────────────────┐
│ Layer 5: 意思決定レイヤー (可視化・レポート)          │
├──────────────────────────────────────────────────┤
│ Layer 4: 形式検証 (Lean 4, Z3) + ZKML              │
├──────────────────────────────────────────────────┤
│ Layer 3: 最適制御 = Neural ODE + Pontryagin         │
├──────────────────────────────────────────────────┤
│ Layer 2: 独立検証 = Diffusion Maps / VDM            │
│          時系列 regime 変化 = Wasserstein distance  │
├──────────────────────────────────────────────────┤
│ Layer 1: 多様体構成 = VAE + pullback                │
│          (小データ・高信頼度領域は GPLVM を選択)      │
├──────────────────────────────────────────────────┤
│ Layer 0: データ品質チェック                          │
└──────────────────────────────────────────────────┘
```

この stacked design の利点:

1. **単一手法の脆弱性を複数手法で相補**: VAE の条件検証失敗時に GPLVM へフォールバック
2. **Layer 2 の独立検証**: 「計量が本当に妥当か」を別方法で確認 (数学者の指摘に直接答える)
3. **Layer 3 が Neural ODE**: GI Vol.3 で Pontryagin 最大原理を first-class citizen として扱える
4. **Layer 4 の形式検証**: 各 layer の計算が数学的に正しいことを Lean 4 で宣言的に検証

---

6. 総合比較 — 数学者から誤りを指摘されないための設計指針

6.1 数学者が指摘する 3 つの典型パターン

実際に微分幾何学・情報幾何学の研究者が社会科学・ML 論文を読むときに指摘する典型パターンを列挙する:

パターン A: 「その計量は真の計量ではない」

> 「あなたが計算している g_{ij}^{VAE} は、デコーダ f_θ に相対的な計量である。真の現象に内在する計量ではない。」

答え方: 正直に認める。本稿 3.5 で述べたとおり、これは model-conditional inference である。

緩和策: 複数手法の一致性をもって「真の計量」に近いことの**必要条件** (ただし十分条件ではない) とする。VAE と Diffusion Map が同じ幾何を抽出するなら、片方のモデル選択依存性を相殺できる。

パターン B: 「滑らかさ / ランクの条件が満たされていない」

> 「ReLU で訓練した VAE では曲率テンソルは定義されない。tanh でも飽和領域で実効ランクが落ちる。」

答え方: 条件 (ii) (iii) の数値検証を常に報告する。Neural ODE を採用すれば構造的に回避可能。

パターン C: 「位相的仮定が間違っている」

> 「潜在空間を $\mathbb{R}^d$ と仮定しているが、データの真の多様体は周期的 / 球面的 / 双曲的である可能性がある。」

答え方: Persistent homology で位相的な事前確認を行う。必要なら Hyperspherical VAE、Torus VAE 等を採用。

6.2 応答テンプレート: 「あなたのフレームワークは数学的に正しいのか?」

数学者から査読・Q&A 等で問われた際の公式応答として、以下のテンプレートを推奨する:

> 「本フレームワークは 4 つの明示的な仮定のもとで数学的に well-defined である。すなわち、
>
> **(i)** 潜在空間 $U \subset \mathbb{R}^d$ はコンパクトな連結集合として扱う **(ii)** デコーダ f_θ の活性化関数は C^∞ 級 (tanh, GELU, Softplus) に限定する **(iii)** ヤコビ行列 J_{f_θ} の最小特異値 $\sigma_{\min}(J) > 10^{-4}$ を解析対象領域の全サンプル点で数値的に検証する **(iv)** 解析対象領域で近似的単射性を N 組の random pair でチェックする
>
> いずれかの条件が違反される領域には **low-confidence フラグ**を付け、その領域での結論は報告しない。これにより、解析結果は 4 条件の充足領域に限定されたものとなる。

さらに、pullback 計量は VAE デコーダに相対的な推定量であり、真の物理的計量の近似である。我々はこの相対性を明示し、独立手法 (Vector Diffusion Maps) による検証を併用して、結論の頑健性を確保する。」

この応答は以下を満たす: - 数学的な主張の範囲を明示 (「全称的に真」ではなく「4 条件下で真」) - 数値検証の手続きを具体的に記述 - 相対性を隠さず、補完的検証を明示

6.3 採用決定のフローチャート

具体的なプロジェクトで「どの手法を採用するか」を決める指針:

```
問題: データから微分幾何学が使える多様体を構成したい
│
├─ Q1: データは大規模 (N > 10^4) か?
│    ├─ YES → VAE または Neural ODE
│    └─ NO  → GPLVM も候補
│
├─ Q2: 不確実性定量化が最優先 (医療・規制対応) か?
│    ├─ YES → GPLVM を優先、または VAE + MC Dropout
│    └─ NO  → VAE 標準
│
├─ Q3: 最適制御・政策シミュレーションが主目的か?
│    ├─ YES → Neural ODE (随伴 = costate の統合性)
│    └─ NO  → VAE で十分
│
├─ Q4: 時系列の分布比較が主目的か?
│    ├─ YES → 最適輸送 (W_2 距離) を主役に
│    └─ NO  → VAE を主役、OT を補完
│
├─ Q5: パラメトリック確率モデルが自然に与えられるか?
│    ├─ YES → 情報幾何学の Fisher 計量
│    └─ NO  → VAE pullback
│
└─ Q6: データの真の位相が $R^d$ でない疑いが強いか?
     ├─ YES → Hyperspherical VAE / Torus VAE / Poincaré VAE
     └─ NO  → 標準 VAE
```

6.4 スタック型設計の再掲と運用原則

本稿 5.8 で示したスタック型設計は以下の運用原則とセットで実装する:

1. **Layer 間の独立性**: 各 layer は単独でユニットテスト可能
2. **早期失敗の原則**: Layer 0 (データ品質) で失敗 → 以降の layer は実行しない

3. **相互検証の義務化**: Layer 1 (VAE) の結果は Layer 2 (Diffusion Map) で常に検証
4. **信頼度の伝播**: 各 layer で信頼度フラグを下流に伝播。最終出力は「信頼度ラベル付き推論」
5. **形式検証のスコープ**: Layer 4 の Lean 4 検証は「アルゴリズムの正しさ」を保証するが、「データ品質」と「モデル適合」は別問題

6.5 リサーチ・アジェンダ

本稿の分析から導かれる未解決課題:

- **RA-1**: Wasserstein 空間上の Ricci 曲率の数値計算アルゴリズム (Sturm-Lott-Villani の具体化)
- **RA-2**: Neural ODE による GI pipeline の完全置換実装。VAE の 4 条件を構造的に回避することの実効的検証
- **RA-3**: Sheaf-theoretic 拡張。GI ケーススタディ 11 の $H^1(B, F)$ を一般的な GI 設定で扱うフレームワーク
- **RA-4**: 動的拡散幾何 (time-varying diffusion map) の理論整備とスペクトル収束
- **RA-5**: データ駆動の位相推定 (persistent homology) と VAE の潜在空間位相の自動選択

Appendix A. 用語集

A.1 微分幾何学

用語	記号	意味
多様体	M	局所的に $\mathbb{R}^d$ と同相な位相空間
接空間	T_pM	点 p における接ベクトル全体。d 次元線型空間
コ接空間	$T_p^* M$	接空間の双対
リーマン計量	g_{ij}	正定値対称双線形形式。距離と角度を定義
pullback 計量	$f^* h$	写像 f で送り返した計量
クリストッフェル記号	Γ_{ij}^k	Levi-Civita 接続の成分

用語	記号	意味
共変微分	$\nabla_X Y$	曲率を考慮したベクトル場の微分
測地線	γ	$\nabla_{\dot\gamma}\dot\gamma = 0$ を満たす曲線
Riemann 曲率	$R^l{}_{ijk}$	∇ の非可換性
Ricci テンソル	Ric_{ij}	Riemann 曲率の縮約
スカラー曲率	Scal	Ricci テンソルの計量縮約
Lie 微分	$\mathcal{L}_V$	ベクトル場のフローに沿った微分
Lie 括弧	$[V, W]$	ベクトル場の非可換性
Killing 場	V s.t. $\mathcal{L}_V g = 0$	等長変換を生成するベクトル場

A.2 位相幾何学

用語	記号	意味
位相空間	$(X, \mathcal{O})$	開集合族を備えた集合
コンパクト	—	任意の開被覆が有限部分被覆を持つ
ハウスドルフ	—	異なる 2 点を分離する開近傍が存在
同相写像	f s.t. f, f^{-1} cts	位相的に同じことを表す
ホモロジー	$H_k(X)$	k 次元の穴を数えるアーベル群
コホモロジー	$H^k(X)$	ホモロジーの双対
層	$\mathcal{F}$	各開集合にデータを割り当てる対応
層コホモロジー	$H^k(X, \mathcal{F})$	グローバル情報取り出し

A.3 最適制御 / 情報幾何

用語	記号	意味
ハミルトニアン	$H(z, p, u)$	Pontryagin 最大原理の関数
随伴変数 (costate)	p	T^*M 上のベクトル
Pontryagin 随伴方程式	$\dot p = -\partial H / \partial z$	最適制御の一次必要条件

用語	記号	意味
Fisher 情報計量	g^{Fisher}	統計多様体上の自然な計量
α -接続	$\nabla^{(\alpha)}$	Amari の接続族
Wasserstein 計量	W_p	確率測度空間上の距離

Appendix B. 参考文献

B.1 数学的基盤

- do Carmo, M. P. (1992). *Riemannian Geometry*. Birkhäuser.
- Hatcher, A. (2002). *Algebraic Topology*. Cambridge University Press.
- Hartshorne, R. (1977). *Algebraic Geometry*. Springer GTM 52.
- Lee, J. M. (2012). *Introduction to Smooth Manifolds* (2nd ed.). Springer GTM 218.

B.2 情報幾何学

- Amari, S. (2016). *Information Geometry and Its Applications*. Springer.
- Amari, S., & Nagaoka, H. (2000). *Methods of Information Geometry*. AMS.
- Chen, R. T. Q., & Murphy, K. P. (2018). Relating VAE to Fisher information.

B.3 最適輸送

- Villani, C. (2009). *Optimal Transport: Old and New*. Springer.
- Peyré, G., & Cuturi, M. (2019). Computational Optimal Transport. *Foundations and Trends in ML*, 11(5-6).
- Otto, F. (2001). The geometry of dissipative evolution equations: the porous medium equation. *Comm. PDE*, 26(1-2), 101-174.
- Feydy, J. et al. (2019). Interpolating between Optimal Transport and MMD using Sinkhorn Divergences. *AISTATS*.
- Weed, J., & Bach, F. (2019). Sharp asymptotic and finite-sample rates of convergence of empirical measures in Wasserstein distance. *Bernoulli*, 25(4A), 2620-2648.

B.4 拡散幾何学

- Coifman, R. R., & Lafon, S. (2006). Diffusion maps. *Appl. Comput. Harmon. Anal.*, 21(1), 5-30.
- Singer, A., & Wu, H.-T. (2012). Vector diffusion maps and the connection Laplacian. *CPAM*, 65(8), 1067-1144.
- Belkin, M., & Niyogi, P. (2008). Towards a theoretical foundation for Laplacian-based manifold methods. *JCSS*, 74(8), 1289-1308.

B.5 Neural ODE / Normalizing Flows

- Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). Neural Ordinary Differential Equations. *NeurIPS*.
- Grathwohl, W., et al. (2019). FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models. *ICLR*.
- Rubanova, Y., Chen, R. T. Q., & Duvenaud, D. (2019). Latent ODEs for Irregularly-Sampled Time Series. *NeurIPS*.

B.6 Gaussian Process / GPLVM

- Lawrence, N. D. (2005). Probabilistic Non-linear Principal Component Analysis with Gaussian Process Latent Variable Models. *JMLR*, 6, 1783-1816.
- Titsias, M. (2009). Variational Learning of Inducing Variables in Sparse Gaussian Processes. *AISTATS*.
- Hensman, J., Fusi, N., & Lawrence, N. D. (2013). Gaussian Processes for Big Data. *UAI*.
- Tosi, A. et al. (2014). Metrics for probabilistic geometries. *UAI*.
- Damianou, A., & Lawrence, N. D. (2013). Deep Gaussian Processes. *AISTATS*.

B.7 非 Euclid 潜在空間 VAE

- Davidson, T. R., et al. (2018). Hyperspherical Variational Auto-Encoders. *UAI*.
- Mathieu, E., et al. (2019). Continuous Hierarchical Representations with Poincaré Variational Auto-Encoders. *NeurIPS*.
- Nickel, M., & Kiela, D. (2017). Poincaré Embeddings for Learning Hierarchical Representations. *NeurIPS*.

B.8 VAE 本体とその幾何学

- Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. *ICLR*.
- Arvanitidis, G., Hansen, L. K., & Hauberg, S. (2018). Latent Space Oddity: On the Curvature of Deep Generative Models. *ICLR*.
- Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian Approximation. *ICML*.

B.9 GI 理論

- Étale Cohomology (2026a). *Geometric Intelligence, Volume 1*. Zenodo. https://doi.org/10.5281/zenodo.19140918
- Étale Cohomology (2026b). *Geometric Intelligence, Volume 2*. Zenodo. https://doi.org/10.5281/zenodo.19157891
- Étale Cohomology (2026c). *偏西風の逆関数 — 12 時間の随伴変数と 248 の境界条件* (GI ケーススタディ・ノベル第 11 作).

Appendix C. 再現性チェックリスト

本稿のコード例を実行する際の再現性チェックリスト:

- ☐ Python 3.10 以上
- ☐ `torch==2.2.0`, `numpy==1.26.4`, `scipy==1.12.0`, `scikit-learn==1.4.0`
- ☐ `torchdiffeq==0.2.3`, `POT==0.9.1`, `pydiffmap==0.2.0.1`, `gpytorch==1.11`
- ☐ ランダムシード固定 (`torch.manual_seed`, `np.random.seed`)
- ☐ CPU での動作確認 (GPU 非必須)
- ☐ 各スクリプトを `python script.py` で単独実行可能

Appendix D. 数学的現状と言責 / Mathematical Caveats and Disclosures

本 Appendix は、本稿の数学的記述について、**厳密な査読に出した場合に指摘されうる箇所**を著者自身が誠実にリストアップしたものである。学術研究では、予想される批判を先回りして明示することが科学的誠実性の核心である。以下の 7 項目は、いずれも本稿全体の結論を覆すものではないが、より厳密な研究論文として発展させる際には修正・補強が必要となる。

D.1 Lie 微分の添字表記の非一貫性

指摘されうる点: 本稿 2.2.6 節で $(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$ と書いた。この式において V_j は**共変ベクトル（1-形式）** であり、V を「ベクトル場」と呼ぶときは通常反変ベクトル（上付き添字）を意味するため、添字位置と座標表現の関係が曖昧である。

厳密な記述: Lie 微分の座標表現は以下である:

$$(\mathcal{L}_V g)_{ij} = V^k \partial_k g_{ij} + g_{kj} \partial_i V^k + g_{ik} \partial_j V^k$$

これを Levi-Civita 接続で書き換えると $\nabla_i V_j + \nabla_j V_i$ となる（ここで $V_j := g_{jk} V^k$、つまり計量で添字を下げた 1-形式）。

影響範囲: 記号の選択の問題であり、数学的内容は変わらない。実装コード（`info_geometry.py` 等）では反変・共変の区別は顕在化しない（計量 g と逆計量 g^{-1} を正しく使っていれば）。

対応: 改訂版では「ベクトル場 V と対応する 1-形式 $V^\flat := g(V, \cdot)$」のように記号を分けて書く。

D.2 Fisher 情報計量の正定値性に必要な identifiability 条件

指摘されうる点: 本稿 4.1.2 節で「p が正則なら g^{Fisher} は正定値対称」と述べた。これは不正確である。

厳密な記述: Fisher 情報行列は常に**半正定値**である。**正定値**（= リーマン計量として非退化）であるためには、以下の **identifiability 条件**が必要:

モデル族 $\{p(x|\theta)\}$ が identifiable である、すなわち
$\theta \neq \theta' \Rightarrow p(\cdot|\theta) \neq p(\cdot|\theta')$ （異なるパラメータが異なる分布を与える）。

これは VAE の Proposition 2.1 における条件 (iv) 単射性の情報幾何版に相当する。

影響範囲: 小さい。多くの応用では identifiability は自動的に成立するか、再パラメータ化で回復できる。混合モデル（ラベル swap による非 identifiability）など特定の場合には注意を要する。

対応: 改訂版では条件を明示する。

D.3 Wasserstein 空間の「リーマン多様体」としての地位

指摘されうる点: 本稿 4.2.1 節で「Wasserstein 空間 $\mathcal{P}_2(\mathbb{R}^n)$ は Otto calculus により形式的にリーマン多様体（無限次元）」と述べた。「形式的に」と留保は付けたが、古典的なリーマン多様体の定義には**完全には収まらない**。

厳密な記述: Wasserstein 空間は以下の性質を持つ:

- **完備距離空間**である（Villani, 2009, Theorem 6.18）
- 絶対連続測度間では**測地線**が一意に存在する（McCann, 1997）
- 接空間は各点 μ で $L^2(\mu; \mathbb{R}^n)/\{\mathrm{gradientfields}\}$ として定義される（Ambrosio, Gigli & Savaré, 2008）
- 滑らかな多様体構造を持つとは限らない；正確には**弱 Riemannian 構造**

「無限次元リーマン多様体」と書くのは文献（Otto, 2001; Lott-Villani, 2009）でも見られるが、査読者によっては「言い過ぎ」と判断されうる。

影響範囲: 本稿の主張（Wasserstein 距離が分布比較の有用ツールである）には影響しない。

対応: 改訂版では「弱 Riemannian 構造」「metric measure space 的構造」と書き換える。

D.4 スペクトル収束定理の条件と引用の対応

指摘されうる点: 本稿 4.3.1 節および 2.3 節で、Belkin-Niyogi (2008) の結果を引用しつつ、Singer-Wu (2012) のレート条件 $N\varepsilon^{d/2+1} \to \infty$ を混在させた。

厳密な記述: 両者は異なる収束の主張をしている:

- **Belkin-Niyogi (2008)**: グラフラプラシアンから Laplace-Beltrami 作用素への**ポイントワイズ収束**。レート条件は $\varepsilon \to 0$, $N\varepsilon^{d/2+1}/\log N \to \infty$ など文献により細部が異なる。
- **Singer-Wu (2012)**: Vector Diffusion Map における接続ラプラシアンへの**スペクトル収束**。レート条件は $N\varepsilon^{d/2+1} \to \infty$。

両者を混在させて引用したのは正確性を欠く。

影響範囲: 本稿の議論（「拡散幾何学は理論的保証がある」）には本質的影響はないが、引用の正確性という学術規範には反する。

対応: 改訂版では各定理の出典と条件を正確に対応させる。

D.5 Proposition 2.1 の「embedded submanifold」と境界の扱い

指摘されうる点: 本稿 1.2 節および 3.1 節で、U がコンパクト（有界閉集合）であるとき $f_\theta(U)$ が d 次元 C^k 級埋め込み部分多様体であると述べた。厳密には、**境界付きコンパクト集合** U に対する埋め込みの像の多様体構造には注意を要する。

厳密な記述:

- U の**内部** $\mathrm{int}(U)$ 上では、$f_\theta|_{\mathrm{int}(U)}$ が単射な C^k 級はめ込みならば、その像は C^k 級埋め込み部分多様体である（Lee, 2012, Proposition 5.2）。

- U の**境界** ∂U 上では、像が「境界付き多様体（manifold with boundary）」の構造を持つかは別議論。
- Lee (2012) Theorem 4.25（本稿で引用）は、コンパクト空間からの単射なはめ込みが埋め込みであることを保証するが、像の多様体構造は内部に限定される。

影響範囲: 実用上、VAE の解析対象領域を $\|z\| \leq 3$ のような有界閉集合にして、**境界から十分離れた内部の領域**でのみ幾何量を計算する場合、影響は小さい。境界近傍での曲率報告は不安定になりうる。

対応: 改訂版では「U を開集合に取り替えて議論する」または「解析は $\mathrm{int}(U)$ に限定する」と明示する。実装では境界からのマージン（例: $\|z\| \leq 3$ ならば $\|z\| \leq 2.8$ の領域のみで曲率を報告）を設ける。

D.6 GPLVM の sample path の滑らかさ

指摘されうる点: 本稿 4.5.2 節で「RBF カーネルなら f は確率 1 で C^∞」と述べた。これは正確には **mean-square sense** での微分可能性であり、**sample path の微分可能性**とは別議論である。

厳密な記述:

- カーネル k が C^{2r} 級ならば、GP の**共分散関数と平均関数**は C^r 級（Paciorek, 2003）
- Sample path の滑らかさには、より強い条件（entropy 条件）が必要（Adler & Taylor, 2007, Section 1.4）
- RBF カーネル（$k(x, x') = \exp(-\|x - x'\|^2/2\ell^2)$）の場合、sample path はほぼ確実に C^∞（これは Adler-Taylor の条件を満たす）
- 他のカーネル（Matérn $\nu = 3/2$ など）では sample path の滑らかさが限定される

影響範囲: 本稿で議論した RBF カーネルの場合には実質的な問題はない。ただし「カーネルの選択によって sample path の滑らかさが変わる」という事実は明示すべきだった。

対応: 改訂版では Adler & Taylor (2007) を引用し、カーネルごとの sample path 滑らかさを明示する。

D.7 Neural ODE の微分同相性の条件

指摘されうる点: 本稿 4.4.2 節および 5.5.2 節で「Neural ODE の f_θ がユニフォーム Lipschitz なら、フロー ϕ_T は C^k 微分同相」と述べた。この主張は**概ね正しい**が、条件の精密化が可能である。

厳密な記述:

- **解の存在と一意性**: f_θ が t と z の両方で連続、かつ z について局所 Lipschitz なら、初期値問題の局所解が一意に存在する（Picard-Lindelöf）
- **大域解の存在**: Grönwall 型の評価で $\|f_\theta(z,t)\| \le C(1+\|z\|)$ （linear growth）なら、解は全時刻で存在する
- **フローの滑らかさ**: $f_\theta \in C^k$ かつ上記の条件を満たせば、ϕ_T は C^k 微分同相（Arnold, 1988, Theorem 2.8.2）

本稿で「Lipschitz」とのみ書いたのは最小条件であり、実際のニューラルネット実装（C^∞ 活性化関数）では C^∞ 微分同相が得られる。

影響範囲: 本稿の結論（Neural ODE が Proposition 2.1 の条件を構造的に保証する）は正しい。より強い保証（C^∞ 微分同相）を明示できていない点が小さな欠落。

対応: 改訂版では Arnold (1988) または Hirsch & Smale の ODE 教科書を引用し、条件を精密化する。

D.8 Manifold hypothesis の経験的地位

指摘されうる点: 本稿全体が **manifold hypothesis**（「データ $X=\{x_i\}\subset\mathbb{R}^n$ がある d 次元多様体 M_0 の近傍からの標本である」）を前提としている。この仮説は経験的仮説であり、数学的定理ではない。

厳密な立場表明:

本稿で提示する全ての幾何学的結論は、**以下の条件付き**で有効である:

1. **(Manifold hypothesis)** データが d 次元連結コンパクト C^∞ 部分多様体 M_0 の近傍からの標本である
2. **(Decoder regularity)** Proposition 2.1 の 4 条件が解析対象領域で数値的に検証されている
3. **(Model-conditional scope)** 結論は「学習されたデコーダ f_θ に相対的」であり、真の現象の内在構造を一意に特定するものではない

反証可能性: 以下の観察は manifold hypothesis の棄却を示唆する:

- Persistent homology が強い Betti 数の非ゼロ性を示す（データに多様体に埋められない位相的特異点がある）
- 異なる ε での Diffusion Map スペクトルが劇的に変化する（内在次元が不安定）
- 異なるランダムシード・訓練データで構成した VAE 多様体間の幾何学的距離が大きい（学習された幾何が不安定）

影響範囲: 本稿の応用（社会経済データの幾何学的分析）の哲学的基盤に関わる。manifold hypothesis が成立しない領域では、本稿の手法全体が機能しない。

対応: 改訂版では最初から「本稿は manifold hypothesis を作業仮説とする」と明示し、反証可能性を認める。この姿勢自体が Popper 的な科学的誠実性の核心である。

D.9 本 Appendix の立場表明

本 Appendix にリストアップした 8 項目は、いずれも:

- **本稿の主要結論を覆すものではない**（代替アプローチの技術的比較、Neural ODE の優位性、スタック型設計の推奨）
- **厳密な研究論文として発展させる際には修正・補強が必要**
- **研究者コミュニティでの議論を通じて改善すべき事項**

科学的研究において、**予想される批判を先回りして明示することは、論文を弱くするのではなく強くする**。本稿を読む数学者・研究者が、これらの点を指摘する必要がなくなり、より建設的な議論に直接進めるようにするのが、本 Appendix の目的である。

Issues, PRs で更なる指摘を歓迎する。修正版は著者の責任で公開する。

D.10 参考文献（Appendix D 専用追加）

- Adler, R. J. (1981). *The Geometry of Random Fields*. Wiley.

- Adler, R. J., & Taylor, J. E. (2007). *Random Fields and Geometry*. Springer.
- Ambrosio, L., Gigli, N., & Savaré, G. (2008). *Gradient Flows in Metric Spaces and in the Space of Probability Measures* (2nd ed.). Birkhäuser.
- Arnold, V. I. (1988). *Geometrical Methods in the Theory of Ordinary Differential Equations* (2nd ed.). Springer.
- McCann, R. J. (1997). A convexity principle for interacting gases. *Advances in Mathematics*, 128(1), 153-179.
- Paciorek, C. J. (2003). *Nonstationary Gaussian Processes for Regression and Spatial Modelling*. PhD Thesis, Carnegie Mellon University.

License

本稿は CC BY 4.0 のもとで公開される。引用時は以下を推奨:

```
@misc{etale2026gi_notes,
  author = {Étale Cohomology},
  title  = {GI 理論の数学的基盤と代替アプローチの比較検討},
  year   = {2026},
  note   = {Research note, draft for open discussion}
}
```

Issues, PRs welcome.